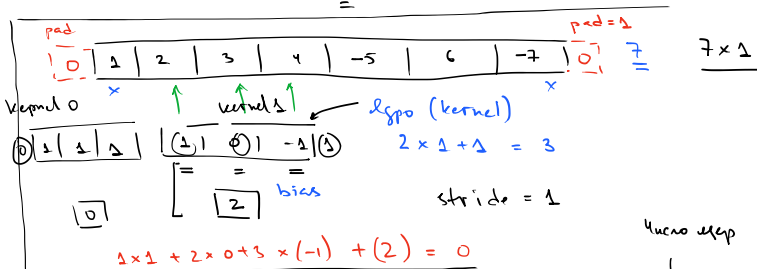


$$200 \times 200 \times 3 \quad (120000 \times 200,000)$$

5

$$200 \times 200 \times 5 = W$$

$$5 \quad b$$



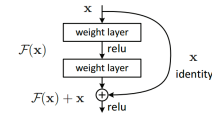
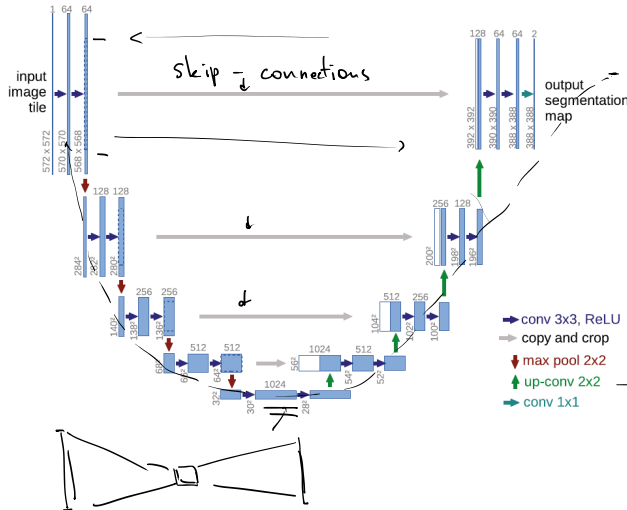
$$(\text{mean-by-channel}, \text{std-by-channel})$$

$$\frac{1}{T} \sum_{t=1}^T$$

INPUT -> [(CONV -> RELU)*N -> POOL?]*M -> [FC -> RELU]*K -> FC

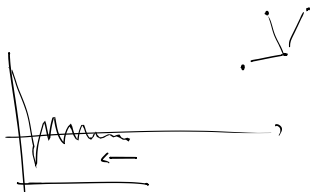
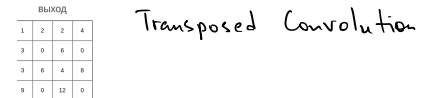
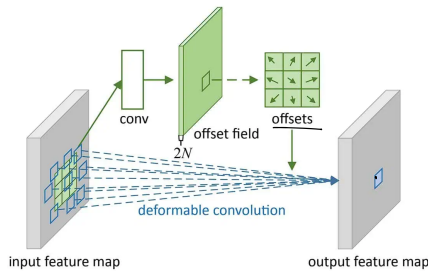
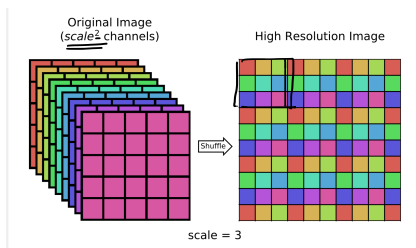
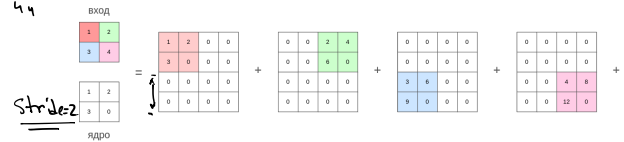
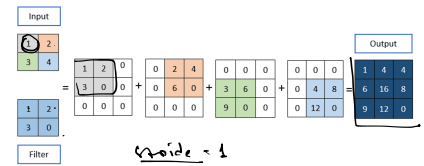
where the $*$ indicates repetition, and the $POOL?$ indicates an optional pooling layer. Moreover, $N \geq 0$ (and usually $N \leq 3$), $M \geq 0$, $K \geq 0$ (and usually $K < 3$). For example, here are some common ConvNet architectures you may see that follow this pattern:

- INPUT -> FC, implements a linear classifier. Here $N = M = K = 0$.
- INPUT -> CONV -> RELU -> FC
- INPUT -> [CONV -> RELU -> POOL]*2 -> FC -> RELU -> FC. Here we see that there is a single CONV layer between every POOL layer.
- INPUT -> [CONV -> RELU -> CONV -> RELU -> POOL]*3 -> [FC -> RELU]*2 -> FC. Here we see two CONV layers stacked before every POOL layer. This is generally a good idea for larger and deeper networks, because multiple stacked CONV layers can develop more complex features of the input volume before the destructive pooling operation.



$$\begin{matrix} 1 & 2 \\ 3 & 4 \end{matrix} \quad \begin{matrix} 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix}$$

$$\begin{matrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 4 & 4 & 4 \\ 3 & 5 & 4 & 4 \end{matrix}$$



$$\frac{1}{T} \sum_{t=1}^T x(t) c(T-t)$$